

02 Algorithmik Asymptotische Analyse

Daniel Gaida

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

17.07.2021

Lernraum I: Abstrakte Datentypen, Datenstrukturen und Asymptotische Notation

- Abstrakte Datentypen
- Liste, Stapel, Warteschlange, Tabelle (Dictionary)
- Datenstrukturen
- Array, Verkettete Liste
- Implementierungen der Abstrakten Datentypen mit den Datenstrukturen ...
- Array
- Verkettete Listen
- Analyse von Laufzeit und Speicherbedarf der Implementierungen
- O-Notation
- Laufzeitanalyse von Algorithmen durch Asymptotische Notation
- O-Notation, Ω -Notation, Θ -Notation
- Fallanalyse: Worst Case, Best Case Analyse
- Laufzeitanalyse von Rekursiven Algorithmen durch Asymptotische Notation

Klassifikation der Wachstumsordnung

Wachstumsordnung: Eine Kategorie der Algorithmeneffizienz, die auf dem Verhältnis des Algorithmus zur Eingabegröße n basiert.

Description

Runtime if you double n
 constant
 unchanged
 logarithmic
 increases slightly
 linear
 doubles
 log-linear
 slightly more than doubles
 quadratic
 quadruples

... 05.02.2026

... Seite 2

exponential

O-Notation

Example Algorithm

$O(1)$

Accessing an index of an array

$O(\log_2 n)$

Binary search

$O(n)$

Looping over an array

$O(n \log_2 n)$

Merge sort algorithm

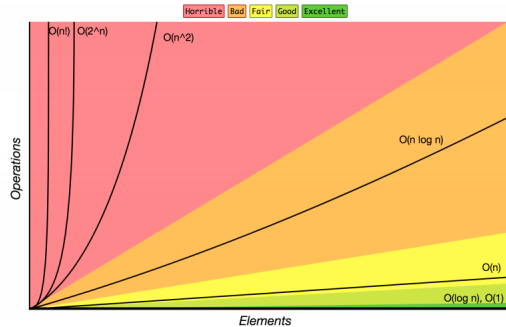
$O(n^2)$

Nested loops!

...

...

$O(2n)$



Quelle: Data Structures and Algorithms (CSE 373), University of Washington, Spring 2022

<https://www.bigocheatsheet.com/>

Code in O-Notation umwandeln

CODE



O-Notation

$O(n)$

```

1  for (i = 0; i < n; i++)
2      {
3          a[i] = 1;
4          b[i] = 2;
5      }

```

Bisher: “ $O(1)$ bedeutet keine Schleifen, $O(n)$ ist eine Schleife, $O(n^2)$ ist verschachtelte Schleife”

- Das ist weiterhin nützlich!
- In diesem Modul gehen wir mehr ins Detail: Damit wir auch komplizierteren Code in die O-Notation umwandeln können

Analyse von Algorithmen

1		2	
CODE		LAUFZEIT FUNKTION	Wachstums- ordnung
	Code Modellie- rung	Asymptotische Analyse	
<pre> 1 for (i = 0; i < n ; i++) { 2 a[i] = 1; 3 b[i] = 2; 4 } </pre>		$f(n) = 2n$	$O(n)$
● Analyse von Algorithmen: Der Gesamtprozess der Zuordnung von Code zu einer Wachstumsordnung, bestehend aus: ● Code Modellierung: Code Laufzeitfunktion, die die Laufzeit des Codes beschreibt ● Asymptotische Notation: Funktion Wachstumsordnung, die das asymptotische Verhalten beschreibt			

Code Modellierung

1

CODE

Code Modellie-
rung

LAUFZEIT
FUNKTION

Code Modellierung - der Prozess der mathematischen Darstellung, wie viele Operationen ein Stück Code im Verhältnis zur Eingabegröße n ausführt. Umwandlung von Code in eine Laufzeitfunktion, die seine Laufzeit darstellt

Code Modellierung Beispiel 2

$$f(n) = (9n+4)n + 3$$

```

1 public int method2(int n) {
2     int sum = 0;
3     int i = 0;
4     while (i < n) {
5         int j = 0;
6         while (j < n) {
7             if (j % 2 == 0) {
8                 // do nothing
9             }
10            sum = sum + (i * 3) + j;
11            j = j + 1;
12        }
13        i = i + 1;
14    } return sum;
15 }
16

```

+1
 +1
 +1
 +1
 +2
 +2
 +4
 +2
 +1

Innere Schleife
 läuft n-mal
 *n*n
 +9

Äußere Schleife
 läuft n-mal
 9n + 4

O-, Ω - und Θ -Notation Übung

O-Notation

$f(n) \in O(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $0 \leq f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$

Welche der folgenden Funktionen ist in $O(n^2)$? $\Omega(n^2)$?

a.

$$f(n) \in O(n^2)$$

b.

$$f(n) \in O(n^2)$$

c.

$$f(n) \in O(n^2)$$

d.

$$f(n) \in O(n^2)$$

$$f(n) \in \Omega(n^2)$$

$$f(n) \in \Theta(n^2)$$

e.

$$f(n) \in \Omega(n^2)$$

Ω -Notation

$f(n) \in \Omega(g(n))$ falls positive Konstanten c und n_0 existieren, sodass
 $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$

Θ -Notation

$f(n) \in \Theta(g(n))$ falls $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$.
 $f(n) \in \Theta(g(n))$ falls positive Konstanten c_1, c_2 und n_0 existieren, sodass
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ für alle $n \geq n_0$

Analyse von Algorithmen

1		2		$O(n)$	
CODE	Code rung	Modellie- rung	LAUFZEIT FUNKTION	Asymptotische Analyse	$\Theta(n)$
<pre> 1 for (i = 0; i < n; i++) { 2 a[i] = 1; 3 b[i] = 2; 4 } 5 </pre>			$f(n) = 2n$	O-Notation Ω-Notation Θ-Notation Quelle: Data Structures and Algorithms (CS-F 373)	
Schauen wir uns dieses Werkzeug nun genauer an. Wie genau kommen wir zu dieser Funktion?		Wir haben dieses Werkzeug fertigge- stellt, um Algorithmen (CS-F 373), einiger nütz- licher Schritte zu charakterisieren!		University of Washington, Spring 2022	
05.02.2026					
Seite 8					

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to the document. If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.